BHEESHMA L
192511332

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO4:
implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 30%

QUESTIONS: 6 **Total Marks:30**

Annexure A

Reverse Engineering Task

Code of Conduct:

I Bheeshma L / 192511332 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
BHEESHMA L

BHEESHMA L
192511332

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

---

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

---

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

BHEESHMA L
192511332

---

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

---

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

---

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

BHEESHMA L
192511332

1. Legacy E-Learning Management System

A. Understanding of the System

The legacy system bundles student registration, course management, content delivery, assignment evaluation, and result generation into large monolithic modules where UI code and business rules are tightly coupled. Any change to one function (e.g., grading rules) risks breaking unrelated features (e.g., registration), and the system cannot be unit-tested easily because business logic is entangled with UI code.

B. Decomposition & Identification of Components

Class / Component	Key Responsibility
Student	Holds student attributes (id, name, enrolledCourses) — domain entity.
Course	Holds course details (code, title, syllabus, instructor).
Content	Represents learning material (video, notes, quiz) linked to a Course.
Assignment / Submission	Represents an assignment and a student's submitted work.
Result	Stores computed grades / scores for a student in a course.
RegistrationService	Business logic for enrolling/unenrolling students.
CourseService	Business logic for creating/updating courses and content.
AssignmentEvaluationService	Grades submissions using pluggable evaluation strategies.
ResultService	Aggregates scores and generates final results.
StudentDAO / CourseDAO / ResultDAO	Persist and retrieve entities from the OODBMS.

C. Reconstruction / Redesign

The system is redesigned into a layered architecture: Presentation Layer (Controllers only handle input/output), Service Layer (RegistrationService, CourseService, AssignmentEvaluationService, ResultService encapsulate all business rules), Domain Layer (Student, Course, Content, Assignment, Result as plain domain objects), and Data Access Layer (DAO classes isolate all OODBMS operations). A Strategy pattern is used inside AssignmentEvaluationService so new evaluation rules (e.g., auto-graded quiz vs. manually graded essay) can be added without modifying existing code.

BHEESHMA L
192511332

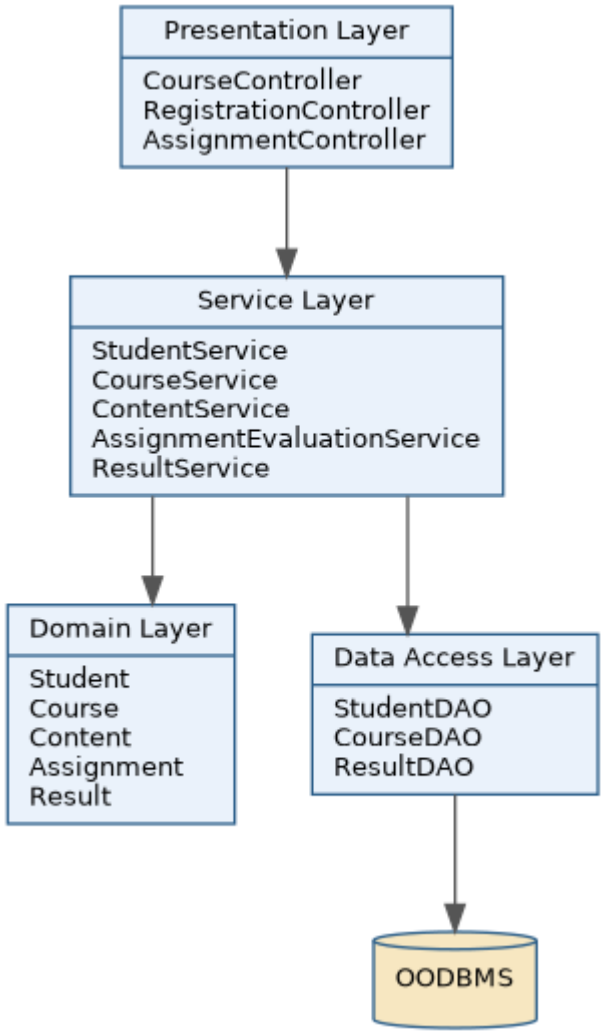

Fig 1.1 — Layered architecture for the redesigned E-Learning system

D. Documentation — Improvements Achieved

- **Cohesion:** each class now has a single, well-defined responsibility (e.g., ResultService only computes/generates results).
- **Coupling:** controllers depend only on service interfaces, and services depend only on DAO interfaces — UI, business logic, and persistence are decoupled.
- **Maintainability:** a change to grading logic touches only AssignmentEvaluationService, not the UI or registration code.
- **Scalability:** new content types, evaluation strategies, or reporting formats can be added by extending, not modifying, existing classes (Open/Closed Principle).

BHEESHMA L
192511332

2. Legacy Customer Support System

A. Understanding of the System

The customer support application uses long conditional (if/switch) blocks to route a request based on channel (email, chatbot, telephone, social media). Because the channel-handling logic lives inside a single class, adding a new channel means editing that class again and again — violating the Open/Closed Principle and risking regression bugs in existing channels.

B. Decomposition & Identification of Components

Class / Component	Key Responsibility
SupportRequest	Domain object holding request details (customer, message, channel).
SupportContext	Delegates handling of a request to the currently configured channel strategy.
SupportChannelStrategy (interface)	Declares handle(request) — common contract for all channels.
EmailSupportStrategy	Concrete handling logic for email-based requests.
ChatbotSupportStrategy	Concrete handling logic for chatbot-based requests.
TelephoneSupportStrategy	Concrete handling logic for telephone-based requests.
SocialMediaSupportStrategy	Concrete handling logic for social-media-based requests.

C. Reconstruction / Redesign

The nested conditionals are replaced with the Strategy behavioral design pattern. Each channel becomes its own concrete strategy class implementing a common SupportChannelStrategy interface. SupportContext holds a reference to the active strategy and simply calls handle(request), remaining unaware of channel-specific logic. A Chain of Responsibility pattern can additionally be layered in for escalation (e.g., bot → human agent → supervisor).

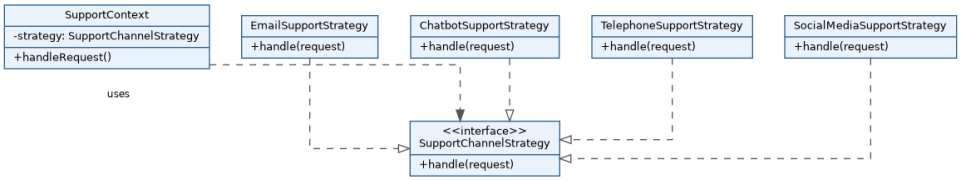

Fig 2.1 — Strategy pattern replacing conditional channel routing

D. Documentation — Improvements Achieved

- **Flexibility:** channel behaviour can be swapped at runtime by changing the strategy reference.
- **Extensibility:** a new channel (e.g., WhatsApp) is added as a new strategy class — no existing class is modified.
- **Loose coupling:** SupportContext depends only on the SupportChannelStrategy interface, not on concrete channel classes.

- **Maintainability:** each channel's logic is isolated, so bugs and changes stay contained to one class.

3. Legacy Banking Transaction System

A. Understanding of the System

Account management, transaction validation, fund transfer, loan processing, and reporting are packed into a few large classes, with database (OODBMS) calls embedded directly inside business methods. This makes the code hard to test (no way to mock the database), hard to reuse, and risky to change since business rules and persistence logic are interwoven.

B. Decomposition & Identification of Components

Class / Component	Key Responsibility
Account, Customer, Loan, Transaction	Domain classes representing core banking entities and their state.
AccountService	Business rules for opening/closing/managing accounts.
TransactionValidationService	Validates a transaction against business rules (balance, limits, fraud checks).
FundTransferService	Coordinates a transfer between two accounts (uses validation service).
LoanProcessingService	Handles loan eligibility, approval, and disbursement logic.
ReportingService	Generates transaction/account reports.
AccountDAO, TransactionDAO, LoanDAO	Isolate all persistence (CRUD) operations from business logic.

C. Reconstruction / Redesign

The system is restructured into a layered architecture: domain classes (pure data + invariants), service classes (business rules, using a Facade-like FundTransferService to coordinate AccountService + TransactionValidationService), and DAO classes (Data Access Object pattern) that hide all OODBMS calls behind an interface. A Unit of Work pattern can wrap a fund transfer so that debit and credit operations commit atomically.

BHEESHMA L
192511332

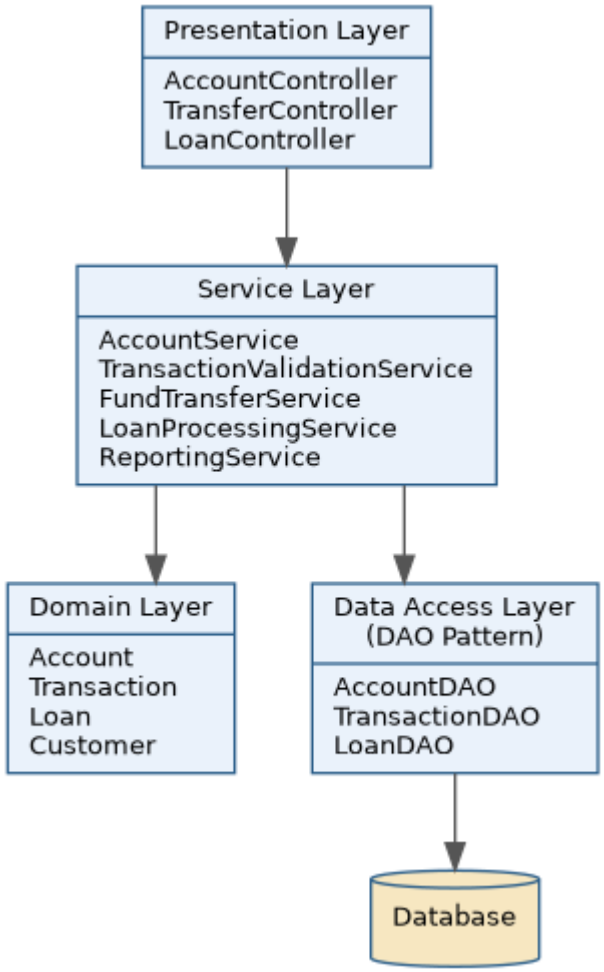

Fig 3.1 — Layered domain/service/DAO architecture for the banking system

D. Documentation — Improvements Achieved

- **Separation of concerns:** validation, transfer, loan and reporting logic each live in their own service class instead of one large class.
- **Testability:** services can be unit-tested by mocking the DAO interfaces, without touching a real database.
- **Maintainability:** a change to persistence technology only affects the DAO layer, not business rules.
- **Coupling:** services depend on DAO abstractions rather than concrete OODBMS calls scattered across methods.

BHEESHMA L
192511332

4. Legacy Hotel Reservation System

A. Understanding of the System

Reservation screens directly issue OODBMS queries, and the same queries for rooms, guests, bookings, and payments are duplicated across multiple modules. This tight coupling between UI and persistence means every screen must know database query syntax, and any schema change requires editing many scattered places.

B. Decomposition & Identification of Components

Class / Component	Key Responsibility
Room, Guest, Booking, Payment	Domain entities representing hotel data.
ReservationService	Business logic for creating/cancelling a reservation.
PaymentService	Business logic for processing and recording payments.
RoomRepository / GuestRepository / BookingRepository (interfaces)	Define find(), save(), delete() contracts for each entity.
OODBMSRoomRepositoryImpl (and siblings)	Concrete implementations that execute the actual OODBMS queries once, in one place.

C. Reconstruction / Redesign

Direct, duplicated OODBMS calls are replaced using the Repository / DAO pattern. Each entity gets a repository interface (RoomRepository, GuestRepository, BookingRepository) and a single concrete OODBMS-backed implementation. Reservation screens now call ReservationService, which uses the repositories — no screen talks to the database directly, and every query for an entity exists in exactly one class.

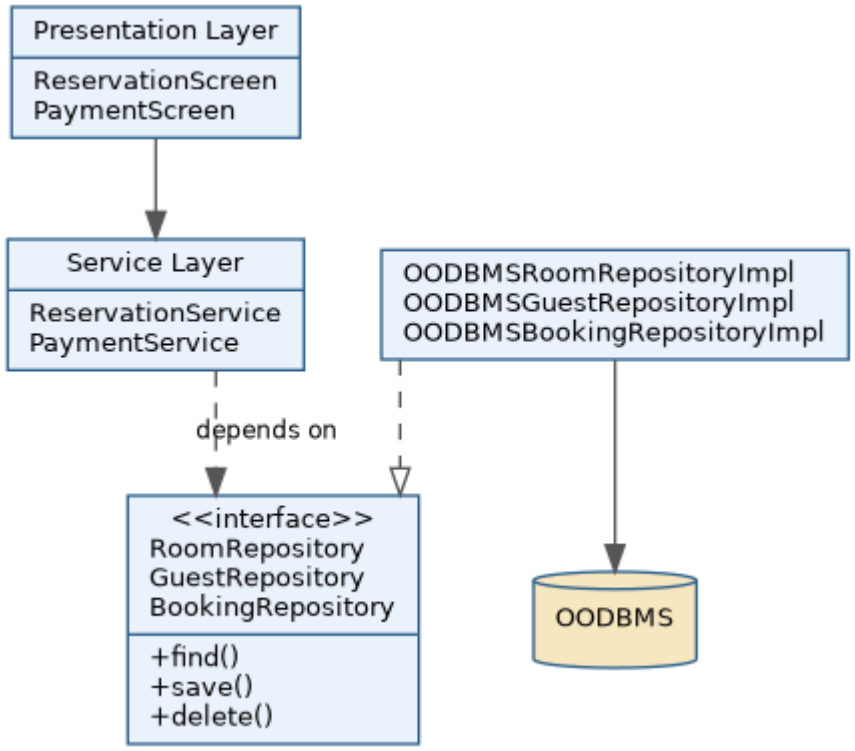

Fig 4.1 — Repository/DAO pattern decoupling screens from the OODBMS

D. Documentation — Improvements Achieved

- **Modularity:** UI, business logic, and persistence are now separate, independently changeable modules.
- **Reusability:** a single RoomRepository is reused by every screen that needs room data, eliminating duplicate queries.
- **OODBMS integration:** database-specific query logic is confined to the repository implementations, so the OODBMS could be swapped with minimal impact.
- **Reduced duplication:** each query is written once and reused, lowering the risk of inconsistent or buggy copies.

5. Legacy Transportation Management System

A. Understanding of the System

Separate modules exist for buses, trains, taxis, and rental vehicles, but the logic to create each vehicle/booking object is duplicated in several places. Adding a new transportation type (e.g., ride-share) means hunting down and modifying every place object creation occurs, which is error-prone and hard to scale.

B. Decomposition & Identification of Components

Class / Component	Key Responsibility
Vehicle (interface)	Common contract (bookSeat(), cancelBooking()) implemented by all transport types.
Bus, Train, Taxi, RentalVehicle	Concrete vehicle types, each with type-specific behaviour.
TransportFactory (interface)	Declares createVehicle() — abstracts object-creation logic.
BusFactory, TrainFactory, TaxiFactory, RentalFactory	Concrete factories, each responsible for creating one vehicle type.
BookingService (client)	Requests a vehicle from the appropriate factory without knowing its concrete class.

C. Reconstruction / Redesign

Common abstractions are identified (Vehicle interface) and object creation is centralized using the Factory Method pattern — one factory per transport type, all implementing a common TransportFactory interface. Where families of related objects are needed (e.g., vehicle + its ticket/invoice type), an Abstract Factory can be layered on top of the Factory Method structure.

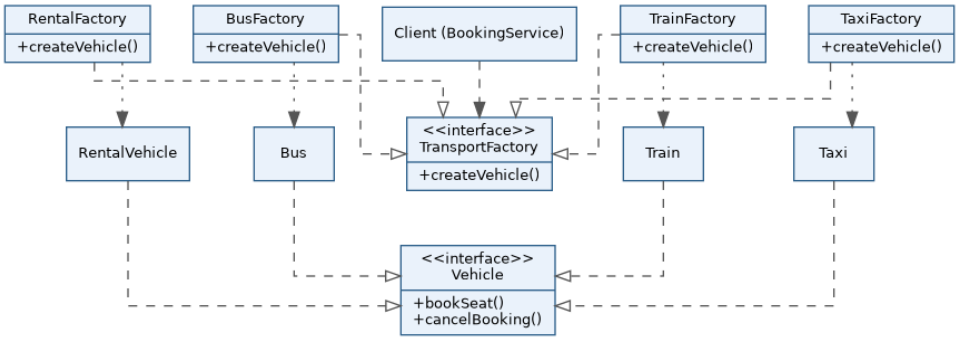

Fig 5.1 — Factory Method pattern for transportation object creation

D. Documentation — Improvements Achieved

- **Extensibility:** a new transport type is added by creating one new Vehicle subclass and one new Factory — no existing code changes.
- **Scalability:** BookingService scales to any number of transport types since it only depends on the TransportFactory abstraction.
- **Code reuse:** common booking/cancellation logic lives once in the Vehicle interface/base behaviour instead of being duplicated per module.
- **Reduced duplication:** object-creation logic is centralized per type instead of scattered across the application.

6. Legacy Healthcare Monitoring System

A. Understanding of the System

The healthcare monitoring application collects patient data, processes sensor readings, generates alerts, stores records, and produces reports — often within the same classes, which directly depend on concrete monitoring-device and storage implementations. This violates the Single Responsibility Principle (SRP — multiple unrelated tasks per class), Open/Closed Principle (OCP — adding a new device/storage means editing existing code), and Dependency Inversion Principle (DIP — high-level logic depends on low-level concrete classes).

B. Decomposition & Identification of Components

Class / Component	Key Responsibility
PatientService	SRP: handles only patient registration and profile management.
SensorProcessingService	SRP: processes raw sensor readings only.
AlertService	SRP: generates alerts based on processed readings.
ReportService	SRP: produces reports from stored records.
MonitoringDevice (interface)	DIP: abstraction that SensorProcessingService depends on instead of a concrete device.
ECGDeviceAdapter, BPDeviceAdapter	Concrete adapters for specific hardware, implementing MonitoringDevice.
MedicalRecordRepository (interface) + OODBMSRecordRepositoryImpl	DIP: abstracts storage so services never depend on a concrete database class.

BHEESHMA L
192511332

C. Reconstruction / Redesign

Each formerly-overloaded class is split by responsibility (SRP). Concrete monitoring devices and storage classes are hidden behind interfaces (MonitoringDevice, MedicalRecordRepository) that high-level services depend on, while concrete Adapter/Impl classes depend on those interfaces (DIP) — dependencies now point toward abstractions, not concrete hardware or storage. New device types or storage engines can be added as new adapter/impl classes without editing existing services (OCP).

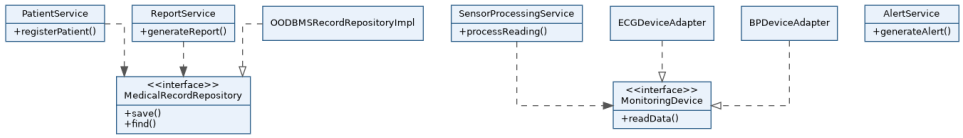

Fig 6.1 — SOLID-compliant redesign using abstractions for devices and storage

D. Documentation — Improvements Achieved

- **Cohesion:** each service now performs one clear task, satisfying SRP.
- **Loose coupling:** services depend on interfaces (MonitoringDevice, MedicalRecordRepository), not concrete classes, satisfying DIP.
- **Flexibility / OCP:** new sensors or storage backends are added as new adapters/implementations without modifying existing services.
- **Testability:** services can be tested with mock devices/repositories instead of real hardware or a live database.
- **Maintainability:** isolated responsibilities make the system easier to understand, debug, and extend.